LLM architecture and training reference

From the 2017 Transformer to October 2026: attention, residuals, MoE, serving, post-training and frontier ideas

12 October 2026

How to use this file: start with Part 2 (Decision guide) when you are choosing what to add to a project, and follow its pointers into Parts 1, 3 and 4 for detail and sources. Part 1 is the timeline, Part 3 covers SFT, preference optimization, RL and distillation, and Part 4 covers recursive, hierarchical, looped, diffusion and memory ideas. Evidence labels (Shipping, Authors-validated, Small-scale research) tell you how far to trust each item.

Contents

- Part 1. LLM architecture timeline: Transformer to October 2026
  - Timeline
  - Attention variants
  - The rest of the stack
  - Study order
  - Unproven or still moving
  - Sources for Part 1
- Part 2. Decision guide: what to add to a model project, and when
  - Evidence labels
  - Architecture checklist by layer
  - By goal
  - By scale
  - Post-training choices
  - Questions that catch missing pieces
  - Not ready yet
- Part 3. Post-training: SFT, preference optimization, RL and distillation
  - The pipeline
  - Supervised fine-tuning and adapters
  - Preference optimization
  - Reinforcement learning algorithms
  - Rewards and verifiers
  - Distillation
  - Test-time compute
  - Failure modes and fixes
  - Frameworks and tools
  - Training-method timeline
  - Sources for Part 3
- Part 4. Components and frontier ideas
  - Recursive language models (RLM)

- Hierarchical and recursive reasoning networks (HRM, TRM)
- Looped and recursive-depth transformers
- Hierarchical and byte-level models
- State-space and hybrid models
- Diffusion language models
- Memory and continual learning
- Position encoding and long context
- Multi-token prediction and speculative decoding
- Training stack parts
- World models and JEPA
- At a glance
- Sources for Part 4

Part 1. LLM architecture timeline: Transformer to October 2026

Since 2017 the big gains came from four directions: cheaper attention, sparse expert layers, rewired residual streams and smarter serving; 2026 models combine all four.

Four parts: Part 1 (this one) is the timeline and attention reference; Part 2, Decision guide, says what to adopt and when; Part 3, Post-training, covers SFT, DPO, PPO, GRPO and distillation; Part 4, Components and frontier ideas, covers recursive, hierarchical, looped, diffusion and memory ideas.

Timeline

Attention compression, expert sparsity and residual rewiring all moved from papers into shipping models between 2023 and 2026. Newest first.

Date	Area	Item	What it changed
2026-10	Memory	Spotlight Memory (Percepta)	Sparse lattice of DeltaNet cells with constant compute per token; results only up to 670M parameters
2026-08	Open coding models	Ornith-1.5 (DeepReinforce)	397B, 35B-A3B and 9B open-weight coding and agent family; scores are vendor-reported
2026-07	Frontier open model	Kimi K3	2.8T-parameter MoE, 16 of 896 experts active; KDA in 3 of every 4 attention layers plus Attention Residuals
2026 (ICLR)	KV quantization	TurboQuant (Google)	Online low-bit KV cache quantizer, reported at 3 bits and roughly 4.6x to 6x smaller, depending on implementation, no calibration
2026-04	Attention, residual, optimizer	DeepSeek-V4	1.6T (49B active) and 284B (13B active) MoE; CSA+HCA attention, mHC, Muon; at 1M tokens uses 27% of V3.2's FLOPs and 10% of its KV cache
2026-03	Residual	Attention Residuals (Kimi)	Softmax attention over earlier layers' outputs replaces the fixed residual sum; block version matched baseline with 20% less compute
2026-01	Memory	Engram (DeepSeek)	Hashed N-gram lookup tables as a second sparsity axis beside MoE; tested at 27B parameters
2025-12	Residual	mHC (DeepSeek)	Constrains hyper-connection mixing matrices to be doubly stochastic; stable training at 3B to 27B
2025-12	Long context	Recursive Language Models (MIT)	Prompt held in a Python REPL; the model recursively calls itself over pieces, handling inputs past 10M tokens
2025-12	Sparse attention	DeepSeek-V3.2	Official release of DSA: a lightning indexer picks the top 2,048 tokens per query (Exp release 2025-09-29)
2025-10	Linear attention	Kimi Linear and KDA	Gated DeltaNet with finer-grained gating; authors report it beats full attention in fair comparisons
2025-10	Distillation	On-Policy Distillation (Thinking Machines)	Student rollouts graded token by token by a teacher with reverse KL; a cheaper alternative to RL
2025-10	Looped models	Ouro	Shared-weight looped transformer; 1.4B and 2.6B models match results of much larger ones
2025-10	Recursive reasoning	Tiny Recursive Model (Samsung)	7M-parameter recursive network that beat HRM on ARC-AGI and Sudoku
~2025-09	Hybrid	Qwen3-Next	Gated DeltaNet layers interleaved with attention layers

Date	Area	Item	What it changed
2025-08	KV retrieval	Spotlight Attention	Non-linear hashing selects critical KV entries; up to 3x decoding throughput; needs about 8 hours of hash training
2025-07	Tokenization	H-Net	Learned dynamic chunking replaces the tokenizer; byte-level model matched a BPE transformer above 1B parameters
~2025-06	Recursive reasoning	Hierarchical Reasoning Model (HRM)	27M-parameter two-timescale recurrent network on ARC-AGI and Sudoku; later ablations credited refinement over hierarchy
~2025-01	Reasoning	DeepSeek-R1	Reinforcement learning (GRPO) on verifiable rewards produced long chain-of-thought
~2024-12	MoE, MLA	DeepSeek-V3	Fine-grained plus shared experts, auxiliary-loss-free balancing, MLA, multi-token prediction
~2024-12	Memory	Titans (Google)	Neural long-term memory updated at test time; largest model 760M parameters
~2024-09	Reasoning	OpenAI o1	First widely used reasoning model; made test-time compute a scaling axis
2024-09	Residual	Hyper-Connections (ByteDance)	Widens the residual stream into parallel lanes with learned mixing
~2024-05	Attention	MLA (DeepSeek-V2)	Compresses keys and values into a low-rank latent to shrink the cache
2023-12	Serving	RadixAttention in SGLang	Radix tree over token prefixes reuses KV cache across requests; up to 6.4x throughput
~2023-12	MoE, SSM	Mixtral 8x7B; Mamba	Open sparse MoE reaches the mainstream; selective state-space model as a transformer alternative
~2023-09	Serving	PagedAttention (vLLM)	Paged KV memory removes fragmentation and raises batch size
~2023-05	Attention	GQA	Query heads share key/value groups; the common dense-model baseline
~2022-05	Kernel	FlashAttention	IO-aware exact attention that fuses softmax with the matmul
~2022-01	Reasoning	Chain-of-thought prompting	Step-by-step prompts raise multi-step accuracy
~2021	Position	RoPE	Rotary position embeddings, still standard (V4 uses it on 64 dimensions)
~2021	MoE	Switch Transformer	Top-1 routing makes trillion-parameter sparse models trainable
2019	Attention	MQA (Shazeer)	All heads share one key/value set; V4's CSA and HCA still use an MQA core
2017-06	Foundation	Transformer (Vaswani et al.)	Self-attention replaces recurrence; cost grows quadratically with length

Rows marked ~ come from my general knowledge or from approximate dates in secondary sources and were not re-opened in full; check them before citing. Dates for training methods (PPO, DPO, GRPO, distillation variants) are in Part 3 (Post-training).

Attention variants

Most attention changes since 2019 shrink the KV cache or skip most of it; the table runs from a full cache to a constant-size state.

Variant	Idea	KV cache effect	Where it appears
MHA (multi-head)	Every head has its own queries, keys and values	Largest cache: one K/V set per head per token	Original Transformer
MQA (multi-query)	All heads share one K/V set	Cache shrinks by the head count	Shazeer 2019; core of V4's CSA and HCA
GQA (grouped-query)	Heads share K/V in groups	Cache shrinks by the group factor	Common dense baseline; V4 paper uses BF16 GQA8 as its reference
MLA (latent)	Keys and values compressed to a low-rank latent	Much smaller cache	DeepSeek-V2, V3, V3.2
Sliding window	Attend only to the most recent tokens	Cache bounded by window size	V4 adds a 128-token window branch to CSA and HCA
DSA (sparse)	Lightning indexer scores past tokens, keeps top-k (2,048 in V3.2)	Full cache kept; reads per token drop	DeepSeek-V3.2
CSA (compressed sparse)	Compress every 4 KV entries into one, then pick top-k (512 Flash, 1,024 Pro)	About 4x shorter, then k reads	DeepSeek-V4
HCA (heavily compressed)	Compress every 128 entries into one, then attend densely	About 128x shorter	DeepSeek-V4
Linear attention, GDN, KDA	Fixed-size recurrent state updated per token	Constant size, but capacity is limited	Qwen3-Next, Kimi Linear, Kimi K3
Spotlight Attention	Hash codes retrieve the critical keys	My reading: cache unchanged, reads drop	Research add-on for existing models
Spotlight Memory	Growing lattice of DeltaNet cells; each token touches 3x3 cells	Grows with allocated cells; access stays constant	Research architecture (Percepta)

DeepSeek's paper puts V4's cache at roughly 2% of a BF16 GQA8 baseline at 1M tokens. CSA and HCA alternate layer by layer, so global recall and cheap long context both survive.

The rest of the stack

Four layers besides attention decide how a 2026 model is built and served: expert routing, reasoning training, the residual stream and the serving path.

- **Mixture of experts (MoE):** each token is routed to a few experts out of many, so active parameters are a small share of the total. DeepSeek-V4-Flash has 1 shared and 256 routed experts with 6 active per token; Kimi K3 activates 16 of 896.
- **Chain of thought and reasoning models:** step-by-step prompting (2022) became reinforcement-learned reasoning (o1, R1), which made test-time compute a scaling axis. V4 ships three effort modes (Non-think, Think High, Think Max), and DeepSeek names long reasoning traces as a reason attention cost had to fall.
- **Residual stream:** a plain sum lets early layers dilute with depth. Two fixes compete: Attention Residuals (attend over earlier layers, in Kimi K3) and mHC (four parallel residual lanes with doubly stochastic mixing, in DeepSeek-V4).
- **Training recipe:** V4 pairs the Muon optimizer with multi-token prediction, on-policy distillation from more than ten expert teachers, and FP4 quantization-aware training for expert weights.
- **Serving:**
  - FlashAttention fuses the attention kernel; PagedAttention pages the KV cache.
  - RadixAttention reuses exact prefixes across requests, so it saves prefill without changing outputs.
  - TurboQuant-style KV quantization shrinks stored bits; Spotlight Attention and DSA skip tokens. Both stack with prefix reuse in principle.

- Compressed layouts break PagedAttention's assumptions, so V4 keeps a separate state cache and stores prefix KV on disk.

Study order

Read in this order, because each step explains a cost that the next step removes.

1. **Transformer (2017):** build a small decoder from scratch and note where memory and compute grow with length.
2. **KV cache and head sharing:** MHA, then MQA, GQA and MLA; compute the cache size for one model by hand.
3. **Kernels and paging:** FlashAttention, then PagedAttention; learn why decoding is memory-bound.
4. **MoE:** Switch Transformer, then DeepSeekMoE; write a top-k router.
5. **Reasoning:** the chain-of-thought paper, then DeepSeek-R1; see how reasoning length drives context cost.
6. **Sparse and compressed attention:** DeepSeek-V3.2's DSA, then sections 2.2 and 2.3 of the DeepSeek-V4 paper.
7. **Linear attention:** Gated DeltaNet, then Kimi Linear and KDA.
8. **Residual rewiring:** Hyper-Connections, then mHC, then Attention Residuals.
9. **Serving systems:** the SGLang paper for RadixAttention, then a KV quantization write-up such as TurboQuant.
10. **Frontier ideas:** Spotlight Attention and Spotlight Memory, reading their baselines critically.

Then read Part 3 (Post-training) in this order: SFT and LoRA, DPO, PPO, GRPO with DAPO and Dr. GRPO, on-policy distillation, failure modes. Finish with Part 4 (Components and frontier ideas), starting with hybrid models and memory.

Unproven or still moving

Several headline results rest on one lab's own numbers or on small models, so treat these as open.

- **Spotlight Memory:** tested only to 670M parameters on FineWeb-Edu, with single-needle recall as the long-context test and no attention-plus-linear hybrid baseline.
- **Attention Residuals:** the 20% compute saving is Moonshot's own measurement; I found no replication by another lab.
- **mHC versus Attention Residuals:** two competing residual redesigns, one in DeepSeek-V4 and one in Kimi K3; I found no head-to-head comparison.
- **CSA indexer cost:** a streaming top-k paper reports that public CSA pipelines run out of memory on one H200 from 64K tokens at V4-Flash dimensions, in a layer-level test on synthetic inputs, because the indexer's score tensor is huge (StreamIndex).
- **Vendor-reported scores:** Ornith-1.5 numbers come from its own runs; Kimi K3's third place on the Artificial Analysis Intelligence Index is an independent measure.
- **Memory benchmarks:** in one study of ten real companion chats, only 3.4% of sampled messages needed anything beyond the current thread (LLM Watch, 11 October 2026), so check what a memory benchmark actually probes.

Sources for Part 1

Pages opened in full this session:

- Attention Residuals, arXiv 2603.15031
- DeepSeek-V4 technical report, arXiv 2606.19348
- Kimi K3 Reveals How A Giant Frontier AI Model Works, The Batch

- Spotlight Attention, arXiv 2508.19740
- Spotlight Memory, Percepta and its companion post, Can LLMs Grow Their Own Capabilities?

Read as search results only, not opened:

- SGLang paper, arXiv 2312.07104
- mHC, arXiv 2512.24880
- DeepSeek-V3.2 in Transformers docs
- Kimi Linear and KDA, arXiv 2510.26692
- Beyond Standard LLMs, Raschka
- TurboQuant overview, Kaitchup
- Ornith-1.5 release, DataNorth
- StreamIndex, arXiv 2605.02568

Part 2. Decision guide: what to add to a model project, and when

Use this part as a checklist: each row names what to adopt, the scale it suits and how much evidence exists. Recommendations are my judgment, built from the sources in the other parts.

Evidence labels

Every row in this part carries one of three labels, so you can tell a proven default from an experiment at a glance.

Label	Meaning	How to treat it
Shipping	A widely used model or serving stack already runs it	Adopt as a default when your constraints match
Authors-validated	Tested at about 1B parameters or more, or against matched baselines, but not shipped	Run a small ablation before committing
Small-scale research	Under about 1B parameters, toy tasks, or a single lab's numbers	Read it, do not build on it

Lines marked "my judgment" are my recommendation from the sources, not a finding in them. Where a recommendation depends on your scale, the table says so.

Architecture checklist by layer

Walk down this table and ask, for each row, whether your project is still on the baseline; the last column says when the upgrade pays.

Layer	Baseline	Upgrade to consider	Evidence	Try it when
Attention heads	MHA	GQA, then MLA	Shipping	KV cache limits batch size or context; GQA is the safe default, MLA needs more pretraining effort
Attention at long context	Full attention	Hybrid linear layers (KDA, Gated DeltaNet), DSA, or CSA with HCA	Shipping	Context passes about 64K; keep some full-attention layers for exact recall
Feed-forward	Dense SwiGLU	MoE with shared plus fine-grained experts and auxiliary-loss-free balancing	Shipping	You want more capacity per FLOP and can handle expert-parallel infrastructure
Residual stream	Plain sum	Block Attention Residuals (Kimi) or mHC (DeepSeek)	Shipping	You pretrain a deep model; see the note below
Normalization	Pre-norm RMSNorm	Per-head RMSNorm on queries and keys	Shipping	Attention logits grow or loss spikes
Position	RoPE	Partial RoPE, scaling for extension, staged length training	Shipping	Extending context past the trained length
Optimizer	AdamW	Muon for matrix weights	Shipping	Pretraining; run both on a small proxy first
Precision	BF16	FP8 training, then FP4 quantization-aware training of experts	Shipping	You control the target hardware and post-training
Output objective	Next token	Add multi-token prediction	Shipping	Pretraining, or you want a built-in drafter
Knowledge storage	Weights only	Engram-style N-gram lookup tables	Authors-validated	Pretraining where knowledge tasks dominate and host memory is cheap
Tokenization	BPE	Byte-level H-Net	Authors-validated	Tokenizer hurts your languages or domain

Layer	Baseline	Upgrade to consider	Evidence	Try it when
Long-input handling	Bigger window, RAG	RLM scaffold with a REPL, or external memory	Authors-validated	Inputs exceed the window or quality drops with length
Decoding	Autoregressive	MTP or speculative drafting; diffusion for latency products	Shipping	Decode latency matters
KV serving	Paged cache	Prefix reuse (RadixAttention), KV quantization, sparse token retrieval	Shipping, Authors-validated for the last two	Shared prefixes, tight memory, very long decodes

Should you add Attention Residuals? My judgment: run it as an ablation, not as a default.

- What it does: each layer attends over earlier layers' outputs with learned, input-dependent weights instead of adding them with equal weight (paper). The block version attends over block-level summaries to cut memory.
- Reasons to try it: Moonshot reports consistent gains across model sizes in scaling-law runs, a 20% training-compute saving for the block version, and it is in Kimi K3 (The Batch); code is public.
- Reasons to wait: no independent replication found; it changes pipeline-parallel communication, so large runs need cache-based communication as the paper describes.
- How to test: two runs at the same token budget, plain residuals against block Attention Residuals, and compare loss, per-layer output magnitude and downstream benchmarks. A third-party Kimi K3 model page lists a block window of 12 layers; treat block size as a hyperparameter to ablate.
- mHC is the competing choice: DeepSeek-V4 uses four residual lanes with doubly stochastic mixing at about 6.7% wall-time overhead; no head-to-head against Attention Residuals exists.

By goal

Start from what you want to improve, take the first move, and only then consider the second.

Goal	First move	Second move	Skip until proven
Longer context	Staged length training with RoPE scaling; hybrid linear plus full-attention layers	CSA and HCA-style compression if you go to 1M	Memory architectures tested only at 670M or less
Cheaper inference	GQA or MLA, prefix caching (RadixAttention), MTP drafting	KV quantization (TurboQuant-style), sparse token retrieval (DSA, Spotlight Attention)	Changing the attention type of a deployed model without retraining
Better reasoning	GRPO with DAPO's dynamic sampling and token-level loss on verifiable tasks	On-policy distillation from a stronger teacher; best-of-N with a verifier	Process reward models unless you can afford step labels
Better agents and tool use	SFT on good trajectories, then step-level agent RL in a sandbox	Harness improvements and an RLM-style REPL for huge inputs	Training the policy before fixing the harness; the same model scored 7 to 17 points higher with a better harness in one benchmark
Quality from a small model	Distill from a stronger teacher; improve data	LoRA adaptation; looped-model experiments if parameters are the limit	Frontier-scale tricks such as FP4 or mHC
Fresh knowledge after training	External memory and retrieval	Periodic adapter updates, then on-policy distillation from the previous version to restore skills	Test-time weight updates in production
Faster generation	MTP or speculative decoding	Diffusion or block-diffusion models	Diffusion for frontier-level reasoning
Languages a tokenizer handles poorly	Extend or retrain the tokenizer	Byte-level H-Net-style models	Betting a product on tokenizer-free models at scale

Goal	First move	Second move	Skip until proven
Training on limited memory	LoRA or QLoRA, gradient checkpointing, mixed precision	Smaller context during training, then extend	Full fine-tuning without a memory budget

The harness figure comes from the Learn2Play benchmark in LLM Watch (11 October 2026), where swapping the harness at a fixed model moved the best score by 7 to 17 points (issue). The restore-skills move after domain adaptation follows the Thinking Machines post, which reports using on-policy distillation to recover instruction-following lost during mid-training (post).

By scale

Most frontier techniques pay off only above a size or context length, so match the upgrade to your budget. This whole table is my judgment from the sources; ablate before you commit.

Scale	Adopt	Test as ablations	Skip for now
Under 1B, pretraining from scratch	RMSNorm, SwiGLU, RoPE, GQA, QK-norm, BF16, AdamW	Muon, Block Attention Residuals, mHC, hybrid linear layers, MTP	FP4, MoE infrastructure, CSA and HCA
1B to 10B, a few GPUs	The above plus a hybrid for long context; LoRA or full SFT; GRPO with a verifier; on-policy distillation from a stronger model	Small MoE, looped layers, Engram-style lookup, MLA	Heavy RL infrastructure
10B to 100B	MoE with shared experts, MLA, FP8, Muon, MTP, hybrid attention, prefix caching	mHC or Attention Residuals, FP4 for experts, sparse attention	Multi-teacher merging until you have several strong specialists
Over 100B	CSA and HCA or DSA, mHC or Attention Residuals, FP4 quantization-aware training, fused MoE kernels, multi-teacher on-policy distillation, deterministic kernels	Engram-style memory, diffusion variants	Test-time memory modules
Fine-tuning an open model (any size)	LoRA or QLoRA SFT, DPO or SimPO, GRPO on checkable tasks, distillation from a stronger model	RLM-style scaffolds, KV quantization at serving time	Changing architecture; you cannot retrofit most of it

Why the line sits where it does: Attention Residuals has scaling-law support across sizes, so cheap ablations are meaningful; V4's compression, FP4 and kernel work needs trillion-parameter, million-token pressure to repay its complexity (V4 report).

Post-training choices

Pick the method from the feedback you actually have: demonstrations, preference pairs, a checker, or a stronger teacher. Details and sources are in Part 3 (Post-training).

What you have	Use	Why
Thousands of demonstrations and one GPU	LoRA or QLoRA SFT	Cheapest behavior change; start every other method from this model
Chosen and rejected pairs	DPO; SimPO if length inflates or memory is tight	Offline, no reward model; variants rarely beat DPO by much
Only thumbs-up or thumbs-down labels	KTO	Needs no pairs
A checker for math, code or tool tasks	GRPO with dynamic sampling and token-level loss	No value model; the working default for RLVR
Outputs that grow long without accuracy gains	Dr. GRPO normalization, overlong penalty	Removes the length bias in GRPO's loss
An MoE model that destabilizes in RL	GSPO, matched trainer and rollout kernels, importance-weight control	Sequence-level ratios and numerics reduce mismatch

What you have	Use	Why
Most prompts all-right or all-wrong	Dynamic sampling, difficulty filtering, self-distillation as an extra signal	Restores gradient where group rewards are uniform
A stronger teacher model	On-policy distillation	Dense per-token feedback; reported far cheaper than RL for the same score
Ground-truth solutions but no teacher	On-policy self-distillation (OPSD)	One model teaches itself from privileged context; matches GRPO on math per its abstract
Several domain expert models	Multi-teacher on-policy distillation	How DeepSeek-V4 merges more than ten experts
Multi-turn agent tasks	Step-level credit methods (GiGPO, ARPO) in a sandbox	Outcome-only rewards give weak signal over long horizons
A model that lost instruction following after domain training	On-policy distillation from the earlier version	Restores skills without redoing RL
Many samples per task at inference	Best-of-N with a verifier; also test the base model's pass@K	RL can lose problems the base model solves given many attempts

Order of operations (my judgment): SFT, then preference tuning or RLVR depending on your feedback, then distillation to consolidate, then add test-time compute; measure pass@1 and pass@K at each step.

Questions that catch missing pieces

Run through these before starting or reviewing a project; each unticked answer points to a row elsewhere in this doc.

- ☐ Is the KV cache my bottleneck? If yes: GQA or MLA, KV quantization, sparse retrieval, prefix caching.
- ☐ Do inputs exceed the window, or does quality fall as they grow? RLM scaffold, external memory, hybrid attention.
- ☐ Do I have a checker for any of my tasks? If yes: RLVR with GRPO instead of preference tuning.
- ☐ Do I have a stronger model that can act as teacher? On-policy distillation before RL.
- ☐ Am I using DPO on a task where a verifier would let the model explore?
- ☐ Do I log entropy, advantage variance and the share of all-right or all-wrong groups from the first step?
- ☐ Does my rollout engine match my trainer numerically, and do I track importance-weight variance?
- ☐ Do I report pass@K next to pass@1, and did I test the base model?
- ☐ Is my residual stream a plain sum in a deep model? Ablate Block Attention Residuals or mHC.
- ☐ Do I use QK-norm, and clamp activations if loss spikes?
- ☐ Am I on AdamW for pretraining without having tried Muon on a proxy?
- ☐ Do many requests share long prefixes? Turn on prefix caching or RadixAttention.
- ☐ Is decode latency the product? MTP, speculative decoding, or a diffusion model.
- ☐ Does a tokenizer hurt my languages or data? Tokenizer work first, byte-level models later.
- ☐ Does my agent re-read huge logs or documents? Harness compaction, RLM-style REPL, external memory.
- ☐ Do I need the model to learn after deployment? External memory plus periodic adapter updates and distillation; not test-time training yet.
- ☐ Do I have a private evaluation set the policy never trains on?
- ☐ Does my memory benchmark actually probe memory? In one real-chat study only 3.4% of messages needed earlier context.
- ☐ Did I choose the harness and the model together, since swapping the harness moved scores by 7 to 17 points in one benchmark?

Not ready yet

Read these, but do not build a product on them until someone shows them at larger scale or against stronger baselines.

- **Spotlight Memory:** tested to 670M parameters, with single-needle recall as the long-context test and no attention-plus-linear hybrid baseline.
- **Titans, HOPE and other test-time memory modules:** the largest Titans model is 760M parameters; HOPE results come from small models.
- **HRM and TRM for language:** strong on puzzles with task-specific training, with no evidence on general text.
- **Unsupervised on-policy self-distillation:** reported on 4B and 8B models and math; confident wrong answers can reinforce themselves.
- **Newer GRPO variants from 2026 (DHPO, TR-GRPO, WAPO, M-GRPO):** limited replication; use DAPO or Dr. GRPO fixes first.
- **Diffusion models for frontier reasoning:** fast and useful for latency products; quality and scalability at the top end are unproven.
- **Attention Residuals as a default:** promising and in Kimi K3, but one lab's measurements; ablate it.
- **Any vendor-reported benchmark you cannot reproduce:** Ornith-1.5, Mercury 2 speed and others; test on your own tasks.

Revisit this list quarterly, and move an item up when a second lab reproduces it or it appears in a shipping model.

Part 3. Post-training: SFT, preference optimization, RL and distillation

Post-training turns a pretrained model into a useful one with four tool families: supervised fine-tuning, preference optimization, reinforcement learning and distillation.

The pipeline

Frontier recipes in 2026 run supervised fine-tuning, then domain reinforcement learning, then on-policy distillation to merge the results.

1. **Pretraining:** next-token prediction on trillions of tokens; DeepSeek-V4 trained on 32T (Flash) and 33T (Pro).
2. **Mid-training and context extension:** raise the sequence length in stages (V4 went 4K, 16K, 64K, 1M) and mix in agentic and reasoning data.
3. **Supervised fine-tuning (SFT):** teach format and behavior from curated demonstrations; the result becomes the reference policy for what follows.
4. **Preference optimization:** DPO-family methods or RLHF for style, helpfulness and safety, where no checker exists.
5. **Reinforcement learning with verifiable rewards (RLVR):** GRPO-family methods for math, code and tool use, where a program can check the answer.
6. **Distillation:** train domain specialists separately, then merge them into one model with multi-teacher on-policy distillation; V4 used more than ten teachers, and Qwen3 closes its pipeline with a reverse-KL distillation pass.
7. **Quantization-aware training and serving:** V4 applies FP4 training to expert weights and the indexer path in post-training so inference matches training.

Sections below cover each family in turn; Part 2 (Decision guide) says which to pick. Entries from my general knowledge are marked approximate.

Supervised fine-tuning and adapters

SFT is the cheapest way to change behavior, and the choice that matters most is how many weights you train and where the demonstrations come from. This section is from my general knowledge, so treat the figures as approximate.

Method	What it trains	Cost	Use it when	Watch for
Full fine-tuning	Every weight	Highest: weights plus optimizer state, several times the model size	You have the GPUs and a large, clean dataset	Forgetting; cost
LoRA	Low-rank adapters on attention and MLP matrices; base frozen	Small	Default for adapting an open model on one or few GPUs	Lower ceiling for big behavior shifts
QLoRA	LoRA on a 4-bit quantized base	Smallest; a 7B-class model fits one consumer GPU	VRAM is the limit	Slower steps; quantization noise
DoRA	Splits each weight into magnitude and direction, adapts direction with LoRA	LoRA-like	LoRA underperforms at the same rank	Small gains
Continued pretraining	Next-token loss on a domain corpus	High	New domain or language	Forgetting general skills

Method	What it trains	Cost	Use it when	Watch for
Teacher-text SFT (sequence-level distillation)	Imitates a stronger model's outputs	Teacher generation cost	Fastest quality jump for a small model	Exposure bias; teacher license terms
Rejection-sampling fine-tuning (expert iteration)	Samples answers, keeps those a checker accepts, trains on them	Needs a verifier	Bootstrapping before RL	Narrow diversity
STaR-style rationales	Keeps self-generated reasoning that reached the right answer	Needs answers	Reasoning from a small seed set	Rewards lucky guesses

Practical rules:

- Mask the prompt tokens from the loss and match the serving chat template exactly.
- Pack short examples into long sequences with attention masks, so no example sees another.
- Mix 5% to 20% general data into narrow SFT to limit forgetting.
- SFT teaches on the teacher's prefixes, not the student's own; on-policy methods below fix that mismatch, which an on-policy self-distillation paper calls exposure bias (OPSD).

Preference optimization

DPO is the default for offline preference tuning of open models, and its variants mostly change the loss shape, the reference term or the data format. In one controlled GSM8K comparison with tuned hyperparameters, DPO reached 58.91 against 56.40 for IPO, 54.59 for KTO and 55.42 for ORPO (PORT); gains from variants are often small and task-dependent.

Method	Feedback it needs	Reference model	What it changes	Use it when
RLHF with PPO	Pairwise preferences, turned into a reward model	Yes, as a KL anchor, plus a value model	Optimizes the policy on-policy against a learned reward	You need maximum control and can afford three or four models
DPO (2023)	Chosen and rejected pairs	Yes	Closed-form loss from KL-constrained reward maximization; no reward model	Default offline method; watch for length bias
IPO	Pairs	Yes	Squared loss with a fixed target margin to curb overfitting	DPO saturates on near-deterministic labels
KTO	Single thumbs-up or thumbs-down labels	Yes	Asymmetric, prospect-theory-style loss; no pairs	Binary feedback is all you have
ORPO	Pairs	No	SFT loss plus an odds-ratio penalty in one stage	You want one-stage training from a base model; more hyperparameter-sensitive
SimPO	Pairs	No	Length-normalized log-probability as the implicit reward, plus a target margin	You want less memory than DPO and less length inflation
Online or iterative DPO	Fresh pairs from the current policy, labeled by a reward model or judge	Yes	Replaces a fixed dataset with on-policy pairs each round	Offline data has gone stale
RLAIF, Constitutional AI	Model-written judgments from written principles	Varies	Replaces human labels with AI labels	You need scale; inherits judge bias

Two cautions apply to all of them:

- Offline methods train only on the comparison distribution, so they cannot explore; where a program can check answers, the RL section below is the better tool.
- Hugging Face's TRL ships maintained trainers for DPO, KTO, ORPO and GRPO, which is a reasonable proxy for the variants that survived 2024 and 2025 (overview). PPO, KTO, ORPO and SimPO details here are from sources opened as search results only.

Reinforcement learning algorithms

GRPO and its variants are the working default for RL on verifiable rewards; they differ in how they normalize advantages, clip updates and weight tokens. When one 2026 paper trained a trillion-parameter model under identical settings, CISPO and DAPO raised reward fastest but were the most prone to entropy collapse and instability, in the order CISPO, DAPO, GRPO; GSPO stayed stable but gave little incentive to produce longer reasoning, which made it a poor fit for bootstrapping reasoning from scratch (Ring-Zero).

Algorithm	Year	Value model	How the advantage is formed	Key change	Best for	Caveat
REINFORCE	1992	No	Reward minus a baseline	Simplest policy gradient	Understanding the family	High variance
PPO	2017	Yes	GAE from the value model	Clipped probability ratio plus KL penalty	RLHF; a safe default if tuned	Extra memory; DeepSeek found it much worse than GRPO at GAE lambda 0.95 and close at 1.0 (R1 paper)
RLOO	~2024	No	Each sample's reward minus the mean of the other samples	Unbiased critic-free baseline	Simple, memory-light RLHF	Needs several samples per prompt
GRPO	2024	No	Reward normalized within a group of samples for the same prompt	Drops the value model; used for DeepSeek-R1	Default RLVR	Length and standard-deviation bias; entropy and advantage collapse
Dr. GRPO	2025	No	Group mean only; no 1/length or standard-deviation division	Removes biases that favor longer wrong answers	Token-efficient long reasoning	Less battle-tested
DAPO	2025	No	GRPO-style	Asymmetric clip (clip-higher), dynamic sampling that skips all-right and all-wrong groups, token-level loss, overlong shaping	Long chain-of-thought RL	More heuristics to tune
GSPO	2025	No	Group-relative	Sequence-level importance ratio matched to sequence-level rewards	Long-response MoE RL	Newer; weak incentive for longer reasoning in Ring-Zero
CISPO	2025	No	GRPO-style	Clips importance-sampling weights so every token, including rare ones, contributes	Off-policy reuse; fast reward growth	Most collapse-prone in Ring-Zero
REINFORCE++	2025	No	Batch-normalized	PPO stabilizers without a critic	Simple baseline	Fewer public comparisons
GiGPO, ARPO	2025	No	Step-level or tool-aware group relative	Credit assignment across turns for tool-using agents	Long-horizon agents	Newer; limited replication
DHPO, TR-GRPO, WAPO, M-GRPO	2026	No	Hybrids and stabilizers (token plus sequence mix, token regulation, positive-only updates, momentum anchor)	Stability against collapse	Experiments	Treat as unproven

What to pick (my judgment): start with GRPO plus DAPO's dynamic sampling and token-level loss; add Dr. GRPO normalization if outputs grow without accuracy gains; try GSPO for MoE models; use CISPO-style correction when rollouts are heavily reused. Sources: DAPO and GRPO comparison, family overview slides, 2026 survey, agent-RL list. Rows marked 1992, ~2024 and REINFORCE++ rest on general knowledge plus these overviews.

Rewards and verifiers

The reward source decides what RL can improve, and a weak reward caps the result more than the algorithm does.

Reward type	What it is	Strength	Failure mode	Example
Verifiable (RLVR)	A program checks the answer: exact match, unit tests, compiler, formal prover	Cheap, hard to fool	Only works where answers are checkable; weak tests get gamed	Math and code RL; DeepSeek-V4 evaluates formal proofs in Lean
Scalar reward model	A network trained on human or AI preferences	Covers subjective quality	Reward hacking, length bias	Classic RLHF
Process reward model	Scores each reasoning step	Dense signal; guides search	Costly labels; style hacking	Step-level search
Generative reward model	An LLM judges with its own reasoning, scored against rubrics	Handles hard-to-verify tasks	Judge bias	V4 trains the policy itself as the judge instead of a scalar model
Rubric or LLM-as-judge	A checklist scored by a model	Open-ended tasks	Judge drift, verbosity bias	Writing, assistant quality
Teacher log-probability	Per-token reverse KL to a teacher, used as the reward	Dense; Thinking Machines argues low KL always means teacher-desirable behavior, so it resists hacking	Needs a teacher	On-policy distillation
Consensus pseudo-labels	Majority vote across the model's own samples	No labels needed	Confident errors reinforce themselves	Label-free self-distillation
Environment outcome	Task success inside a sandbox	Real agent skill	Sandbox cost; sparse reward	V4's sandbox platform runs hundreds of thousands of instances

Rules of thumb (my judgment): prefer a verifier over a model whenever one exists; if you need a model judge, test it against held-out human labels first; keep a small private evaluation set the policy never sees. Sources: V4 report, On-Policy Distillation, GRPO production failures.

Distillation

On-policy distillation became the closing post-training stage at several frontier labs by late 2025 and 2026: one tutorial lists Qwen3, GLM-5, MiMo-V2 and Nemotron-Cascade 2 (tutorial). The student samples its own outputs and a teacher grades every token, which gives dense feedback without the distribution mismatch of copying teacher text.

Type	Student learns from	On-policy	Signal	Loss	Notes
Sequence-level (teacher-text SFT)	Teacher-written answers	No	Dense, but only on the teacher's prefixes	Cross-entropy	Cheapest start; exposure bias; teacher license applies
Logit distillation	The teacher's full next-token distribution on fixed text	No	Dense	Forward KL (mean-seeking)	Needs teacher logits and a shared tokenizer
Reverse-KL distillation (MiniLLM, GKD)	Teacher scores of student samples	Yes	Dense	Reverse KL (mode-seeking)	Bolts onto an existing GRPO trainer
On-policy distillation (Thinking Machines, 27 Oct 2025)	Per-token teacher grades of student rollouts	Yes	About one bit of feedback per generated token versus one per episode for outcome-reward RL	Per-token reverse KL as the advantage	Reported 74.4% versus 67.6% on AIME 2024 against RL, at 1,800 versus 17,920 GPU hours, reproducing Qwen3's table; other sources cite 10x to 100x savings
Multi-teacher on-policy distillation	Weighted reverse KL to many expert teachers	Yes	Dense	Full-vocabulary logits	DeepSeek-V4 merges more than ten domain experts into one student (report)
On-policy self-distillation (OPSD)	One model: the teacher copy sees the ground-truth solution, the student sees only the problem	Yes	Dense	Reverse-KL style	ICML 2026; matches GRPO and beats SFT on four competition-math tasks per its abstract
Unsupervised OPSD (Aug 2026)	Teacher conditioned on a majority-vote pseudo-solution	Yes	Dense	As OPSD	+8.5% and +10.7% over the base model at 4B and 8B (Qwen3 non-thinking); small-scale, math

Type	Student learns from	On-policy	Signal	Loss	Notes
DualOPSD (Aug 2026)	Privileged teacher that co-evolves with the student	Yes	Dense	Alternating updates	Fixes the frozen-teacher assumption; early
Self-Retrospection Distillation (SRD)	The agent's own hindsight from a finished trajectory, distilled into the pre-attempt policy	Yes	Dense where rewards are uniform	Auxiliary loss on top of RLVR	Reported up to 24.2 points over RLVR; a 2B run went from 0.0% to 60.6% success; only the abstract was available to the newsletter (LLM Watch)
Context or prompt distillation	A teacher that is the same model plus a long prompt or scaffold	Either	Dense	KL	Moves harness text into weights to save tokens (general knowledge)
Feature distillation	Intermediate activations of the teacher	No	Dense	MSE or cosine	Needs matching architecture; older BERT-era approach (general knowledge)
Drafter distillation	A small drafter trained to match the target model	Yes	Dense	KL	Raises speculative-decoding acceptance (general knowledge)

Choosing (my judgment):

- A stronger teacher exists: on-policy distillation usually beats teacher-text SFT and costs less than RL; use a truncated or top-k reverse KL for stability, as a March 2026 paper proposes (Revisiting OPD).
- Only a verifier exists: GRPO-family RL.
- Ground-truth solutions exist but no teacher: try OPSD.
- Nothing external exists: unsupervised OPSD is promising but so far shown only on small models and math.
- Many specialists exist: merge them with multi-teacher distillation rather than averaging weights.

Test-time compute

Spending more compute at inference can substitute for training, but only when something can tell good answers from bad ones.

- **Best-of-N with a verifier:** sample N answers and keep the one a checker or reward model prefers; the simplest large gain wherever a verifier exists.
- **Majority vote (self-consistency):** cheap, but agreement can hide shared errors. Google's VeriHarness checks rollouts with the same base model, resolves disagreements against environment evidence and challenges shared claims; it added 6.2 points over a single rollout with Gemini 3.5 Flash and 6.4 with Claude Opus 4.8 on five long-horizon benchmarks (weekly roundup).
- **Step-level search:** beam search or tree search guided by a process reward model; costly to label, strong on math.
- **Sequential revision:** the model critiques and rewrites its answer; gains depend on a reliable critic.
- **Reasoning effort control:** V4 ships three modes, Non-think, Think High and Think Max, trained with different length penalties and context windows (8K, 128K and 384K tokens in its evaluations).
- **Base model plus light harness at large K:** Meta, Wisconsin and Stanford found that across 42 base and post-trained pairs, RL post-training often lost problems the base model could solve given many attempts; the "Sharpening Tax" was positive in 36 of 42 cases and an 8-rollout estimate predicted it (LLM Watch). If your product samples many candidates, test the base model too.
- **Recursive decomposition:** the model splits a huge input and calls itself on the pieces; see recursive language models in Part 4 (Components and frontier ideas).

Budget rule (my judgment): add test-time compute after RL, not instead of it, and measure pass@K alongside pass@1 so you can see whether training narrowed what the model can solve.

Failure modes and fixes

These eight patterns cover the RL breakdowns reported in the sources below; log entropy and advantage variance from the first step.

Failure	Symptom	Likely cause	Fix
Entropy collapse	Entropy drops early; policy turns overconfident; reward plateaus; calibration worsens	Tokens with positive advantage dominate updates; off-policy updates; narrow data; tight clipping (entropy study)	Clip-higher, reweight positive-advantage tokens, entropy bonus, more diverse prompts
Advantage collapse	Whole groups get the same reward, so the gradient is zero	Prompts too easy or too hard	Dynamic sampling, difficulty filtering, curriculum; auxiliary signals such as self-distillation (in one study 37% to 98% of groups were reward-uniform)
Length bias or runaway length	Outputs grow without accuracy gains	Per-sample length normalization and standard-deviation scaling	Dr. GRPO normalization, token-level loss, overlong penalty
Training-inference mismatch	Importance weights between trainer and rollout engine diverge; MoE runs destabilize	Different kernels or precision in the two stacks; router drift (MoE study)	Sequence-level ratios (GSPO), importance-weight clipping (CISPO), matched or batch-invariant kernels
Reward hacking	Reward climbs while real quality falls	Exploitable verifier or reward model	Harder checks, private held-out evaluation, KL anchor, generative or rubric judges
Tool-use RL collapse	Abrupt reward collapse when training agents from a base model	Low-likelihood wrong answers give huge importance weights (lazy likelihood displacement)	Stabilized GRPO variants, start from SFT, step-level credit
KL drift or format loss	Outputs degrade; format and calibration from SFT disappear	KL coefficient too low	Tune KL, start RL from a strong SFT model
Sharpening tax	pass@1 rises, pass@K falls	RL pushes each task toward always or never solved	Per-prompt temperature (PTGS), route between base and post-trained models

DeepSeek-V4's two stabilizers for trillion-parameter MoE training are worth knowing: anticipatory routing, which computes routing indices from earlier parameters and cost about 20% extra time when active, switched on only after a loss spike; and clamping the SwiGLU linear component to the range -10 to 10 (report). It also built bitwise batch-invariant, deterministic kernels so training and inference match exactly.

Frameworks and tools

Prototype on a single-GPU library, then move to a distributed RL framework when rollouts or model size outgrow it.

Tool	Use it for	Notes
TRL (Hugging Face)	SFT, DPO, KTO, ORPO, GRPO and PPO trainers	Gets new alignment methods early; broad multi-GPU support; not speed-optimized
Unsloth	Single-GPU LoRA and QLoRA	Claims about 2x speed and 30% to 70% less memory; works with TRL trainers; added MoE training in February 2026
Axolotl, LLaMA-Factory	Config-driven fine-tuning stacks	Wide model coverage; LLaMA-Factory adds a web UI
OpenRLHF	PPO, GRPO, RLOO and DPO on Ray with vLLM rollouts	Handles heterogeneous node placement; a single framework for the family
verl (ByteDance)	Large-scale RL, up to hundreds of billions of parameters with Megatron	Strongest at 70B and above per one comparison (infrastructure guide)
vLLM, SGLang	Rollout generation during RL	Prefix reuse helps when many samples share a prompt (general knowledge)

Tool	Use it for	Notes
Tinker cookbook	Reference recipes for on-policy distillation	Includes the reverse-KL setup from the Thinking Machines post

Setup order (my judgment): SFT with TRL or Unsloth on one GPU; DPO or SimPO next if you have preference pairs; GRPO with a verifier in TRL for small models; move to OpenRLHF or verl only when rollouts dominate your wall-clock time.

Training-method timeline

The center of gravity moved from reward models and PPO (2022) to critic-free RL (2024 to 2025) to dense, on-policy distillation (2025 to 2026). Newest first; rows marked ~ are from my general knowledge.

Date	Method	What it added
2026-08	Unsupervised OPSD and DualOPSD	Label-free and adaptive-teacher on-policy self-distillation
2026-07	Ring-Zero	Same-setting comparison of GRPO, DAPO, CISPO and GSPO at a trillion parameters
2026-03	Revisiting OPD	Truncated reverse KL with top-k support matching and special-token masking
2026-01	OPSD (ICML 2026)	One model as teacher and student via privileged ground-truth context
2025-10	On-Policy Distillation	Per-token reverse-KL grading of student rollouts as a cheap alternative to RL
~2025-07	GSPO (Qwen)	Sequence-level importance ratios for stable long-response MoE RL
~2025-06	CISPO (MiniMax-M1)	Clipped importance-sampling weights so rare tokens still train
~2025-03	DAPO (ByteDance), Dr. GRPO	Clip-higher, dynamic sampling, token-level loss; bias-free normalization
2025-01	DeepSeek-R1	GRPO with verifiable rewards at scale; showed critic-free RL can produce long reasoning
~2024-02	GRPO (DeepSeekMath)	Group-normalized advantages that remove the value model
~2024-02	KTO, then ORPO and SimPO	Unpaired feedback; reference-free preference losses
2023-05	DPO	Preference tuning without a reward model or RL loop
~2022	InstructGPT and Constitutional AI	RLHF with PPO at scale; AI-written preference labels
2017	PPO (Schulman et al.)	Clipped surrogate objective that became the RLHF workhorse

Sources for Part 3

This part rests on search results and newsletter text, not on pages opened in full; the V4 report was opened in full earlier (Part 1).

- Ring-Zero, arXiv 2607.12395
- DeepSeek-R1, arXiv 2501.12948
- DAPO and Dr. GRPO discussion, Interconnects
- RL algorithm family slides, Berkeley
- Reasoning RL in 2026, Turing Post
- PORT: DPO variant comparison, arXiv 2406.16061
- DPO, arXiv 2305.18290
- On-Policy Distillation, Thinking Machines

- On-policy distillation tutorial
- Revisiting On-Policy Distillation, arXiv 2603.25562
- OPSD, arXiv 2601.18734
- Unsupervised OPSD, arXiv 2608.06296
- DualOPSD, arXiv 2608.26019
- Entropy in RLVR, arXiv 2511.05993
- Objective-level hacking in MoE RLVR, arXiv 2602.01103
- Lazy likelihood displacement in agent search RL, arXiv 2512.04220
- GRPO in production failure modes
- Agent-RL algorithm list, arXiv 2606.12384
- LLM Watch and Top AI Papers of the Week, 11 October 2026 (links in Part 1 and above)
- Fine-tuning frameworks compared, OpenRLHF paper, RLHF infrastructure guide

Part 4. Components and frontier ideas

This part catalogs architecture ideas beyond the standard transformer, from recursive and hierarchical models to memory, diffusion and training-stack parts, each with its evidence level.

Recursive language models (RLM)

An RLM is an inference scaffold, not a new network: it moves a huge prompt out of the context window into a Python environment and lets the model call itself on pieces of it.

- **How it works:** the prompt becomes a variable in a persistent REPL; the model writes code to inspect and split it, calls a sub-model or sub-RLM on the pieces, and stitches the results (Zhang, Kraska, Khattab, MIT, submitted 31 December 2025, revised 11 May 2026).
- **Evidence:** the authors report handling inputs beyond 10 million tokens, past GPT-5's 272K window, with double-digit gains over base models and common long-context scaffolds on four tasks at comparable or lower cost per query; a fine-tuned RLM-Qwen3-8B beat base Qwen3-8B by a median of 28.3% (summary).
- **Why it works:** it avoids context rot, the drop in quality as inputs grow dense, by separating storage (the REPL), access (code) and reasoning (the model).
- **Use it when:** inputs exceed the window or quality degrades on long, information-dense inputs, and you already run a tool-using agent.
- **Caveats:** recursive calls need depth and budget limits; results rest on four benchmarks; it adds latency and orchestration code, and a weak model writes poor decomposition code.
- **How to add it (my judgment):** give an existing agent a REPL with the document loaded as a variable and a `llm_query(snippet)` function, cap recursion depth at 2 or 3, and log calls per task.

Hierarchical and recursive reasoning networks (HRM, TRM)

HRM and TRM are tiny recurrent networks that solve puzzles by refining an answer many times; they are strong on structured puzzles and untested as language models.

- **HRM (Hierarchical Reasoning Model, 2025):** 27M parameters, a slow high-level and a fast low-level recurrent module, trained from about 1,000 examples; reported 40.3% on ARC-AGI-1 (overview). The ARC Prize team's ablations found the gain came mostly from refining the answer across supervision steps, not from the hierarchy (TRM summary).
- **TRM (Tiny Recursive Model, Samsung, October 2025):** one two-layer network that keeps a current answer and a latent scratchpad and alternates updating them; about 7M parameters, 44.6% on ARC-AGI-1 and 7.8% on ARC-AGI-2 (report). Removing deep supervision dropped ARC-AGI-1 accuracy from 39% to 19%; removing the hierarchy cost about 3 points (analysis).
- **Use it when:** the task is a bounded, grid-like or search-like puzzle (Sudoku, mazes, ARC-style) and you have a small, heavily augmented training set.
- **Caveats:** results use task-specific training data with puzzle identifiers and augmentation, so they do not compare cleanly with general LLMs; a technical note on the released TRM checkpoint attributes its ARC-AGI-1 score to an interaction of efficiency and task-specific design (arXiv 2512.11847).
- **Takeaway for your own models (my judgment):** the transferable idea is iterative refinement with supervision at every step, not the brain-inspired hierarchy.

Looped and recursive-depth transformers

Looped models reuse one block of layers several times per token, trading extra compute for fewer parameters and treating loop count as a third scaling axis beside size and data.

- **Ouro (LoopLM):** a shared block applied up to four recurrent steps, an entropy-regularized objective that learns how many steps each input needs, and 7.7T training tokens; the abstract says the 1.4B and 2.6B models match results of up to 12B models (arXiv 2510.25741, submitted 29 October 2025, revised 1 July 2026).
- **What loops add:** controlled experiments found recurrence does not raise knowledge storage (about 2 bits per parameter either way) but improves fact composition and multi-hop reasoning; the authors also report reasoning traces more faithful than explicit chain-of-thought (paper).
- **Relatives:** Mixture-of-Recursions routes tokens to different recursion depths; recurrent-depth models scale test-time compute in latent space; a hybrid latent attention paper for looped models appeared in early October 2026 (paper list).
- **Use it when (my judgment):** parameters or memory are the limit but compute is cheap, such as on-device or small-GPU serving, and the task needs multi-step reasoning more than stored facts.
- **Caveats:** per-token compute and latency rise with loop count; an earlier version of the Ouro paper itself called frontier-scale gains unproven; in my searches I found no frontier lab shipping a looped model.
- **Cheap experiment (my judgment):** tie the weights of the middle third of an existing small transformer, run it two or three times, and compare reasoning benchmarks at equal FLOPs.

Hierarchical and byte-level models

H-Net learns where to split raw bytes into chunks inside the model, replacing the tokenizer with a trained, content-dependent chunker.

- **How it works:** a dynamic chunking mechanism learns segmentation jointly with the rest of the network, so a small byte-level encoder feeds a larger model that works on chunks, and a decoder expands back to bytes (H-Net, arXiv 2507.07955, July 2025).
- **Evidence:** with one stage of hierarchy, a byte-level H-Net matched or beat a strong BPE-tokenized transformer at sizes above 1B parameters when compute- and data-matched, with better character-level robustness and gains on Chinese and code (MIT talk abstract).
- **Earlier steps:** MegaByte, SpaceByte and Byte Latent Transformer introduced patching and data-dependent chunking; H-Net generalizes them end to end.
- **Variants:** H-Net++ adapts the idea to morphologically rich languages, with chunks that align with Persian morphology (arXiv 2508.05628).
- **Use it when (my judgment):** a tokenizer hurts you, for example low-resource or morphologically rich languages, code, noisy text or non-text sequences such as DNA.
- **Caveats:** the largest reported models are slightly above 1B parameters; training infrastructure is less mature than for token models; serving stacks assume token sequences.

State-space and hybrid models

The shipping pattern is a hybrid: mostly cheap fixed-state layers with a minority of full-attention layers kept for exact recall.

Model or idea	Mix	What to note
Mamba (~2023), Mamba-2	Selective state-space layers, no attention	Linear cost, constant state; weaker at exact recall when used alone (general knowledge)

Model or idea	Mix	What to note
Jamba (~2024)	Mamba, attention and MoE layers interleaved	First widely known large hybrid (general knowledge)
Qwen3-Next and Qwen3.5	Gated DeltaNet linear attention interleaved with attention	Gated DeltaNet updates a running memory with the delta rule
Kimi Linear and KDA	Kimi Delta Attention, a finer-gated Gated DeltaNet	Cut memory by up to 75% and sped output up to 6x at 1M tokens in Kimi Linear; Kimi K3 uses it in 3 of every 4 attention layers
Nemotron 3 (NVIDIA)	Mamba-2 layers with transformer layers	Nemotron 3 Ultra: 550B total, 55B active, 1M context (overview)
MiniMax Lightning Attention	Linear attention, O(n) cost	Same overview
Spotlight Memory	DeltaNet cells in a growing lattice	Research; see the memory section below

- **Why hybrids:** pure fixed-state layers run out of capacity; in Percepta's test Gated DeltaNet fell to 0.005 recall at 131K key-value pairs, while attention stayed perfect (post).
- **Use it when:** context is long and serving cost matters; start near 3 linear layers per full-attention layer, as Kimi K3 does, and tune from there (my judgment).
- **Caveats:** kernels and serving support lag standard attention; hybrid KV caches need custom layouts, as DeepSeek-V4's cache design shows; benefits shrink at short context.

Diffusion language models

Diffusion LMs generate many tokens in parallel by repeatedly denoising masked text, and by 2026 they are commercial products where speed matters more than frontier reasoning.

- **Open models:** LLaDA scaled masked diffusion to 8B parameters and was competitive with similar-size autoregressive models; LLaDA 2.0 scaled to 100B and LLaDA 2.1 added token editing (Kuleshov group overview).
- **Hybrids:** Block Diffusion (ICLR 2025) models sequences block by block with diffusion inside each block, which restores flexible length and KV caching.
- **Commercial:** Inception's Mercury 2, launched 24 February 2026, reportedly reached 1,009 tokens per second on one Blackwell GPU, over 5x faster than Claude 4.5 Haiku Reasoning at about 89 tokens per second (secondary report); Google and NVIDIA also have diffusion models, and NVIDIA's Nemotron-Labs-Diffusion unifies autoregressive, diffusion and self-speculation decoding in one model (arXiv 2607.05722).
- **Post-training:** RL and preference methods exist (d1, VRPO) and on-policy self-distillation for diffusion models appeared in 2026, but the tooling is far less mature than for autoregressive models.
- **Use it when (my judgment):** latency is the product (autocomplete, code editing, infilling) or you need controllable, non-left-to-right generation.
- **Caveats:** a survey notes scalability is underexplored against autoregressive models; speed claims are vendor-reported; frontier-level reasoning and long-context quality are unproven.

Memory and continual learning

Memory ideas split into three kinds: memory that is learned at test time, memory that is looked up, and memory that lives outside the model; none of the first two has reached a frontier model.

Idea	Mechanism	Evidence reported	Largest scale	Status
Titans and MIRAS (Google)	Neural long-term memory updated at test time by a surprise signal; MIRAS frames designs by memory architecture, attentional bias, retention gate and learning rule	Beat Transformer++, Mamba-2 and Gated DeltaNet in the paper's tests; strong on BABILong	760M parameters	Research
ATLAS	Optimizes memory over a context window rather than one token at a time	Improves on Titans and reports +80% accuracy at 10M context on BABILong, per its abstract	Small	Research
Nested Learning and HOPE (NeurIPS 2025)	Self-modifying Titans variant with memories updating at different speeds	Beat Titans, Samba and a Transformer on language modeling and continual-learning tests (explainer)	Small	Research
TTT-E2E (NVIDIA)	Compress the context into weights by next-token prediction at test time	Constant latency; 2.7x faster than full attention at 128K and 35x at 2M on an H100	Not stated here	Research
Engram (DeepSeek, January 2026)	Hashed N-gram embedding tables looked up in O(1) and fused into deep layers	At 27B, beat an iso-parameter, iso-FLOPs MoE: BBH +5.0, ARC-Challenge +3.7, HumanEval +3.0, MATH +2.4, multi-query needle 84.2 to 97.0 (summary); a 100B table offloads to CPU for under 3% overhead	27B	Authors-validated; not in V4's published design
Spotlight Memory (Percepta, October 2026)	Growing 2D lattice of DeltaNet cells; each token touches a fixed 3x3 block	Near-perfect recall at 131K pairs; 93% to 100% single-needle recall at 128K after 8K training	670M parameters	Research
Continuous Memory Machine (Sakana, October 2026)	Short- and long-term matrix memories read and written by a Transformer	Beats LSTM, DNC and CTM baselines on copy, recall, sorting, maze tasks	Toy tasks	Research
External agent memory	Stores, retrievers and summaries outside the model	Treated as its own architectural layer with its own benchmarks in 2026 (Mem0 survey)	Production	Shipping

- **Engram versus V4:** DeepSeek-V4's report lists CSA and HCA, mHC and Muon as its upgrades and does not mention Engram, so treat Engram as published research, not a shipped component.
- **Use it when (my judgment):** for knowledge lookup in a model you pretrain, Engram is the best-evidenced new idea here because it was tested at 27B with matched parameters and FLOPs; for long context in a model you fine-tune, use external memory plus RLM-style access first.
- **Caveat:** in one study of ten real companion chats only 3.4% of sampled messages needed earlier context, so check what a memory benchmark actually probes (LLM Watch).

Position encoding and long context

A long-context model needs a position scheme that extrapolates, a training schedule that grows length in stages, and a way to stop attention logits from exploding.

Technique	What it does	Where it appears	Note
RoPE (rotary embeddings)	Rotates queries and keys by position-dependent angles	Nearly every 2026 open model	DeepSeek-V4 applies it to only the last 64 dimensions of each head, and also applies the opposite rotation to attention outputs so they carry relative position
YaRN, NTK scaling	Rescales RoPE frequencies so a model trained short can run longer	Context extension of pretrained models	Needs a short fine-tune to hold quality (general knowledge)
NoPE and mixed layers	No positional embedding in some or all layers	Some recent designs	Works only with enough training length diversity (general knowledge)

Technique	What it does	Where it appears	Note
Attention sinks	Learnable logits that let a head attend to nothing	DeepSeek-V4 adds a learnable sink per head	Keeps total attention from being forced to 1
Sliding-window branch	Exact attention over the most recent tokens	V4 adds a 128-token window to CSA and HCA	Recovers local detail lost to compression
Staged length training	Train at short length, then extend	V4 went 4K, 16K, 64K, 1M, with dense attention for the first 1T tokens before introducing sparse attention at 64K	Introduce sparsity late, and warm up the indexer first
QK normalization	RMSNorm on queries and keys before attention	V4 does this on every head; Kimi's QK-Clip is the alternative	Prevents exploding logits, so V4 skips QK-Clip

Checklist for extending context (my judgment): pick RoPE with a scaling method, train the new length in stages with a mix of long documents, evaluate with needle and aggregation tasks (RULER, LongBench-style) rather than perplexity alone, and test beyond the trained length, since an architecture can extrapolate to 16x its training length, as Spotlight Memory's authors report. Source for V4 details: report.

Multi-token prediction and speculative decoding

Multi-token prediction (MTP) trains a model to predict several future tokens, and speculative decoding uses a cheap drafter to propose tokens that the full model verifies in one pass.

- **MTP in practice:** DeepSeek-V3 and V4 add an MTP module at depth 1 with loss weight 0.3 for most of training and 0.1 once the learning rate decays (V4 report); the extra head can double as a built-in drafter at inference.
- **Speculative decoding:** a draft proposes k tokens, the target scores them in parallel, and a rejection rule keeps the output distribution identical to normal decoding, so quality does not change (general knowledge).
- **Drafter families:** a separate small model, extra heads on the target (Medusa), feature-level drafters such as EAGLE and its successors, and self-speculation, where one model drafts for itself; Nemotron-Labs-Diffusion supports autoregressive, diffusion and self-speculation modes in one model (arXiv 2607.05722).
- **Complication:** compressed attention makes the cache depend on the whole prefix, so tree-structured speculation on DeepSeek-V4 needs the candidate tree, causal visibility and the evolving compressed history to stay consistent (arXiv 2609.24698).
- **Use it when (my judgment):** decode latency matters and batch sizes are small; the gain shrinks at large batch sizes, where decoding becomes compute-bound.
- **Pairs with:** drafter distillation (see Part 3, Post-training) to raise the acceptance rate, and KV quantization or sparse attention to cut memory traffic.

Training stack parts

DeepSeek-V4's report is the most complete public recipe for a 2026 frontier model, so the table below uses it as the reference point.

Part	Common baseline	2026 option	Evidence
Optimizer	AdamW	Muon for most weights, using 10 hybrid Newton-Schulz steps; AdamW kept for embeddings, the output head, RMSNorm weights and mHC biases	V4 report; faster convergence and better stability claimed
Numeric precision	BF16	FP8 training; FP4 quantization-aware training of MoE expert weights and the CSA indexer path	V4: FP4-to-FP8 dequantization is lossless; index scores in BF16 gave a 2x top-k speedup at 99.7% recall

Part	Common baseline	2026 option	Evidence
Normalization	RMSNorm, pre-norm	Per-head RMSNorm on queries and compressed keys before attention	V4; removes the need for QK-Clip
Activation	SwiGLU	SwiGLU with the linear part clamped to -10 to 10 and the gate capped at 10	V4; removed outlier-driven loss spikes
MoE design	Top-k routed experts	Shared plus fine-grained routed experts, auxiliary-loss-free balancing with a small sequence-level balance loss, hash routing in the first MoE layers	V4: 1 shared and 384 routed experts (Pro), 6 active
MoE communication	Separate all-to-all, then compute	A fused wave-scheduled kernel that overlaps dispatch, compute and combine	V4: 1.50x to 1.73x faster inference, up to 1.96x for RL rollouts; open-sourced as MegaMoE
Residual stream	Plain residual sum	mHC or Attention Residuals	V4 mHC costs 6.7% wall-time overhead
Instability control	Rollback and restart	Anticipatory routing and SwiGLU clamping	V4
Reproducibility	Nondeterministic kernels	Bitwise batch-invariant and deterministic kernels across training and inference	V4; also helps RL, where trainer and rollout must agree
Stability tricks without a reference	z-loss, muP scaling, warmup	Standard practice (general knowledge)	Common in open recipes

Source: DeepSeek-V4 technical report. Use it as a checklist of questions rather than a shopping list: each row has a cheaper small-scale equivalent (AdamW, BF16, RMSNorm, SwiGLU, standard MoE), and Part 2 (Decision guide) says which upgrades are worth it at which scale.

World models and JEPA

Joint-embedding predictive architectures predict abstract representations of future states instead of pixels or tokens, and they remain a research direction rather than a component you can adopt.

- **Idea:** Yann LeCun's JEPA learns from images, video and other signals by predicting future latent states with self-supervised learning; it sits inside an agent that has perception, memory, an actor and a cost module (CACM, October 2026).
- **Related lines:** state-space models for long sequences, agentic architectures that decompose problems, and neural memory modules appear in the same survey as post-transformer directions; VL-JEPA and LLM-JEPA are 2025 to 2026 refinements (overview).
- **Status:** I found no frontier language model built on JEPA; the practical near-term path in these sources is hybrids that combine transformers with state-space, diffusion or JEPA components.
- **Use it when (my judgment):** you work on embodied agents, video or planning in latent space; for text products, spend effort on the attention, memory and post-training rows instead.

At a glance

Fourteen of the twenty-two rows below already ship in a major model or serving stack; the rest are research or add-ons. Evidence labels: **Shipping** means a widely used model or serving stack uses it; **Authors-validated** means tested at roughly 1B parameters or more, or against matched baselines, but not shipped; **Small-scale research** means under about 1B parameters or toy tasks.

Idea	Problem it solves	Evidence	Adopt when
GQA	KV cache size	Shipping	Default for any dense model
MLA	KV cache size with high quality	Shipping	You pretrain and serve long context at scale

Idea	Problem it solves	Evidence	Adopt when
DSA (sparse attention)	Attention cost at long context	Shipping	Context beyond about 64K, and you can train an indexer
CSA and HCA	KV cache and FLOPs at 1M tokens	Shipping	Million-token context is a product requirement
Hybrid linear attention (KDA, Gated DeltaNet, Mamba-2)	Cost and memory at long context	Shipping	Long context, with some full-attention layers kept
Attention Residuals	Residual dilution with depth	Shipping	Pretraining a deep model; 20% compute saving is the authors' figure
mHC	Stable wider residual stream	Shipping	Pretraining at scale, with infrastructure for the 6.7% overhead
Muon	Faster, more stable training	Shipping	Pretraining; test against AdamW on your scale first
FP4 quantization-aware training	Memory and speed at inference	Shipping	You control post-training and the target hardware
MTP	Training signal and built-in drafting	Shipping	Pretraining, or decode latency matters
RadixAttention	Repeated prefixes across requests	Shipping	Agents, RAG, chat with shared system prompts
GRPO-family RL	Reasoning and tool use with checkers	Shipping	A verifier exists
On-policy distillation	Merging specialists, shrinking models	Shipping	A teacher exists
Diffusion LMs	Decode speed	Shipping (speed-focused)	Latency is the product
TurboQuant-style KV quantization	KV cache memory	Authors-validated	Serving long contexts on limited memory
Spotlight Attention	Token selection from the cache	Authors-validated (add-on)	Existing model, long decoding, training budget for hashing
Engram	Knowledge lookup without compute	Authors-validated (27B)	Pretraining a model where knowledge tasks dominate
Ouro-style looped models	Parameter efficiency for reasoning	Authors-validated (up to 2.6B)	Parameter or memory limited, compute cheap
H-Net	Tokenizer-free modeling	Authors-validated (above 1B)	Tokenizers hurt your languages or data
RLM	Context rot on huge inputs	Authors-validated (inference scaffold)	Inputs exceed the window now; no retraining needed
Titans, HOPE, TTT-E2E	Test-time memory and continual learning	Small-scale research	Research projects only
Spotlight Memory, HRM, TRM	Growing memory; puzzle reasoning	Small-scale research	Research projects only

Sources for Part 4

Opened in full: the DeepSeek-V4 report and the Kimi K3 article from Part 1, and the Percepta posts you supplied. Everything else here was read as search results or newsletter text.

- DeepSeek-V4 technical report
- Kimi K3 Reveals How A Giant Frontier AI Model Works, The Batch
- Spotlight Memory, Percepta
- Recursive Language Models, arXiv 2512.24601 and summary

- HRM overview, TRM summary, TRM report, TRM analysis, TRM checkpoint note, arXiv 2512.11847
- Ouro, arXiv 2510.25741 and daily paper list, 8 October 2026
- H-Net, arXiv 2507.07955, H-Net++, arXiv 2508.05628, MIT talk abstract
- Beyond Standard LLMs, Raschka, Top 9 LLMs, October 2026
- How to build a diffusion language model, Kuleshov group, Mercury 2 report, Nemotron-Labs-Diffusion, arXiv 2607.05722
- Titans and MIRAS, Google Research, Nested Learning explainer, TTT-E2E, NVIDIA
- Engram, DeepSeek and summary
- Speculative decoding on DeepSeek-V4, arXiv 2609.24698
- Beyond LLMs: A Post-Transformer World Emerges, CACM, The End of LLMs As We Know Them
- State of AI Agent Memory 2026, Mem0, LLM Watch, 11 October 2026